Documentação do Projeto — Tech Challenge Fase 04

Documento de entrega: arquitetura do sistema, guia de uso e relato de experiências e desafios.

---

1. Visão geral

O projeto entrega a interface mobile da aplicação de blogging construída ao longo do curso. O app é escrito em **React Native com Expo** e consome a API REST em **Node.js/Express** criada na Fase 02, que nesta fase foi estendida com autenticação e com o cadastro de professores e alunos.

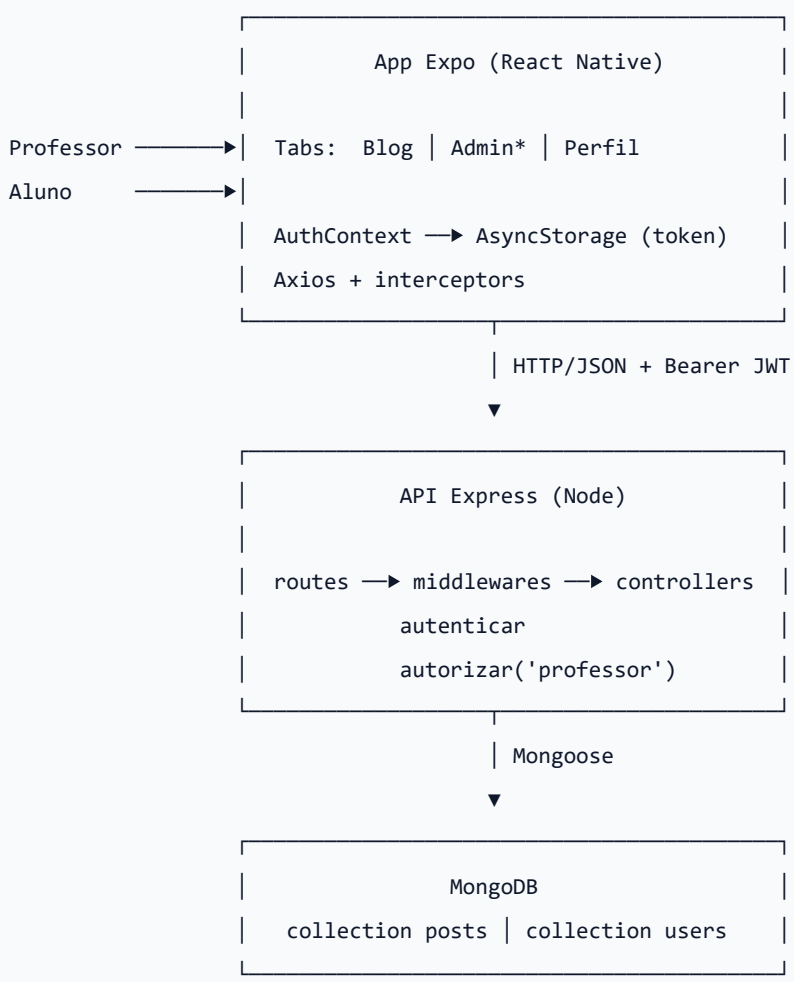
O repositório é um monorepo com duas aplicações:

Pasta	Aplicação
`backend/`	API REST (Node + Express + Mongoose + MongoDB)
`mobile/`	Aplicativo React Native (Expo)

---

2. Arquitetura do sistema

2.1 Diagrama de componentes



* a aba Admin só é montada quando o usuário logado tem role = professor

2.2 Backend

Camadas. `routes` definem a superfície HTTP; `middlewares` resolvem autenticação, autorização e tradução de erros; `controllers` concentram a regra de negócio; `models` descrevem os schemas do Mongoose. Nenhum controller conhece o `req.headers` — quem resolve identidade é o middleware.

Modelo de dados.

```
User                                Post
├── nome        String (req)        ├── titulo    String (req, máx 150)
├── email       String (req, único) ├── conteudo  String (req)
├── senha       String (req, hash)  ├── autor     String (req)
├── role        'professor' | 'aluno' ├── autorId   ObjectId → User
├── materia     String (professor)  └── timestamps
├── turma       String (aluno)
└── timestamps
```

Professores e alunos vivem na mesma collection, separados por `role`. A alternativa seria duas collections quase idênticas; com um modelo só, as regras de senha, validação de e-mail e serialização ficam escritas uma vez, e as rotas `/professores` e `/alunos` nascem da mesma fábrica de controller (`criarControllerDeUsuario`), que apenas fixa o `role` no filtro e na criação.

Segurança.

- Senha com hash **bcrypt** (10 rounds) aplicado num hook `pre('save')` — vale tanto para criação quanto para troca de senha na edição.
- O campo `senha` tem `select: false` e é removido no `toJSON`; não existe caminho em que ele apareça numa resposta.
- **JWT** assinado com `{ id, role }` e validade configurável (`JWT_EXPIRES_IN`, padrão 7 dias).
- `autenticar` valida o token e **recarrega o usuário no banco** — um usuário excluído não continua acessando com um token antigo.
- `autorizar('professor')` protege toda a escrita de posts e todo o CRUD de usuários.
- Regra extra: o professor logado não consegue excluir a própria conta (evita deixar o sistema sem administrador).

Contrato de resposta. Toda rota devolve `{ success, ... }`. Listagens incluem `count` e um bloco `meta` com `page`, `limit`, `total`, `totalPages`, `hasNextPage` e `hasPrevPage`, consumido diretamente pelo componente de paginação do app.

2.3 Mobile

Navegação (`src/navigation/RootNavigator.js`):

```
Root (stack)
├── Tabs (bottom tabs)
│   ├── Blog (stack)    → PostList · PostDetail · PostForm
│   ├── Admin (stack)   → AdminHome · AdminPosts · PostForm · PostDetail
│   │                     ProfessorList · ProfessorForm
│   │                     AlunoList · AlunoForm
│   └── Perfil (stack)  → PerfilHome
└── Login (modal)
```

A aba **Admin** não é escondida por CSS: ela simplesmente não é montada quando `role !== 'professor'`. Quem não é professor não tem rota para chegar lá.

Estado global. `AuthContext` (Context API + hooks) guarda o usuário autenticado, persiste token e perfil no `AsyncStorage`, revalida a sessão em `GET /auth/me` na abertura do app e limpa tudo quando a API responde 401. Foi escolhido Context API em vez de Redux porque o estado global do app se resume a "quem está logado" — Redux adicionaria boilerplate sem resolver nenhum problema que o app tenha.

Camada de rede (`src/api/client.js`): uma instância Axios com

- interceptor de *request* que injeta `Authorization: Bearer <token>` ;
- interceptor de *response* que, em 401, dispara o logout registrado pelo `AuthContext` ;
- `extrairMensagemDeErro` , que converte erro de validação, timeout e `Network Error` em uma frase que o usuário entende — inclusive citando a URL da API quando a conexão falha, que é o erro mais comum em ambiente de desenvolvimento.

Reuso de telas. `UserListScreen` e `UserFormScreen` atendem professores e alunos. As rotas continuam separadas (`ProfessorList` , `AlunoForm` , ...) e passam `role` via `initialParams` ; a tela troca títulos, rótulos e o campo específico (matéria/turma). Isso cumpre o requisito 8 do enunciado sem duplicar quatro telas.

Hook de listagem. `useListaPaginada` concentra paginação, busca com *debounce* de 400 ms, *pull-to-refresh*, estado de erro e descarte de respostas fora de ordem (guarda o id da requisição e ignora respostas antigas). As três listagens do app usam esse mesmo hook.

Design system. `src/theme` define cores, espaçamentos, raios, tipografia e sombra; os componentes (`AppButton` , `AppInput` , `PostCard` , `SearchBar` , `UserRow` , `Paginador` , `Feedback`) consomem apenas esses tokens. Trocar a cor primária do app é mudar uma linha.

2.4 Decisões de arquitetura

Decisão	Alternativa descartada	Motivo
Uma collection `users` com `role`	Collections separadas de professores e alunos	Evita duplicar schema, hash de senha e seis handlers CRUD
Context API	Redux	O estado global é só a sessão; Redux seria peso sem retorno
React Navigation (stack + tabs)	Expo Router	É a biblioteca apresentada nas aulas da fase
Paginação no servidor	Carregar tudo e paginar no app	O requisito 7 pede listagem paginada e o tráfego cresce com a base
Tela genérica por `role`	Quatro telas específicas	Mesmo comportamento com um quarto do código a manter
Expo	React Native CLI	Dispensa Android Studio/JDK para desenvolver e abre o app por QR code

---

3. Guia de uso

3.1 Subindo o ambiente

Instruções completas no README. Resumo:

```
# API + banco
docker compose up -d --build
docker compose exec api npm run seed

# ou, sem Docker:
cd backend && npm run dev:memory

# app
cd mobile && npm start
```

3.2 Perfis e permissões

Ação	Visitante	Aluno	Professor
Ver lista de posts e buscar	✅	✅	✅
Ler post completo	✅	✅	✅
Criar / editar / excluir post	❌	❌	✅
Painel administrativo	❌	❌	✅
CRUD de professores e alunos	❌	❌	✅

As três colunas são garantidas no backend. O app só reflete a regra escondendo o que o usuário não pode fazer.

3.3 Fluxos principais

Ler um post. Aba *Blog* → busca ou rolagem → toque no card → conteúdo completo.

Publicar. Login como professor → botão flutuante **+** na aba *Blog* (ou *Admin* → *Postagens* → *Nova postagem*) → título, autor e conteúdo → **Publicar post**.

Editar/excluir post. *Admin* → *Postagens* → botões no rodapé de cada card. A exclusão pede confirmação.

Cadastrar professor ou aluno. *Admin* → *Professores* (ou *Alunos*) → **Novo** → nome, e-mail, matéria/turma e senha. Na edição, deixar a senha em branco mantém a atual.

Sair. Aba *Perfil* → **Sair da conta**.

---

4. Relato de experiências e desafios

> Esta seção descreve o que foi enfrentado na construção. Ajuste os trechos marcados com *[grupo]* com a experiência de vocês antes da entrega.

4.1 O backend da Fase 02 não atendia a Fase 04

O primeiro diagnóstico foi o mais importante: o enunciado pede login de professores, CRUD de docentes e CRUD de estudantes, mas a API herdada só tinha `Post` — sem usuários, sem `/auth`, sem papéis. O login do front web da fase anterior era uma comparação de string no próprio cliente (`if (username === 'professor' ...)`), o que não autentica nada.

Foi preciso, então, estender a API antes de escrever qualquer tela: modelo `User`, hash com bcrypt, emissão e validação de JWT, middlewares de autenticação e autorização, e os dois conjuntos de rotas administrativas. O aprendizado aqui foi perceber que **o requisito 10 ("apenas usuários autenticados...") é um requisito de backend**: esconder o botão no app não protege o endpoint.

4.2 Projeto retomado em outra máquina

O desenvolvimento desta fase começou em um computador diferente do usado nas fases anteriores, sem Docker, sem JDK e sem Android SDK instalados. Isso gerou duas decisões úteis:

1. **Modo** `npm run dev:memory`: um ponto de entrada que sobe a API com `mongodb-memory-server` já populada, sem exigir Docker nem MongoDB instalado. Serve para desenvolver, testar e gravar a demonstração.
2. `.env.example` **documentado** nos dois projetos, para que qualquer integrante clone e rode sem adivinhar variável.

4.3 `localhost` não existe dentro do emulador

Um erro clássico e difícil de diagnosticar na primeira vez: o app no emulador Android apontando para `http://localhost:3001` falha com `Network Error`, porque `localhost` ali é o próprio emulador, não a máquina que roda a API. A solução foi resolver a URL por plataforma no `client.js` (`10.0.2.2` no Android, `localhost` no resto) e permitir sobrescrever por `EXPO_PUBLIC_API_URL` para quem testar em celular físico pela rede.

Para tornar o problema visível quando acontecer, a mensagem de erro do app cita a URL que ele tentou usar, em vez de um "Network Error" seco.

4.4 Evitar quatro telas quase iguais

O requisito 8 manda replicar as telas de professores para alunos. A tentação era duplicar os arquivos. Optamos por telas genéricas parametrizadas por `role`, mantendo rotas separadas: a navegação continua legível (`AlunoForm`, `ProfessorList`) e existe um só lugar para corrigir um bug de formulário. O mesmo raciocínio foi aplicado no backend, com a fábrica de controllers.

4.5 Busca que não briga com a digitação

A primeira versão da busca disparava uma requisição por tecla, e as respostas chegavam fora de ordem — digitar "react" rápido podia terminar exibindo o resultado de "rea". A correção foi concentrar isso no `useListaPaginada`: *debounce* de 400 ms, reset para a página 1 a cada novo termo e um contador de requisição que descarta respostas que não são a mais recente.

4.6 Testes como rede de segurança

A suíte do backend cresceu de 1 para 22 testes, cobrindo os caminhos que o app depende: login válido e inválido, token ausente/inválido, leitura pública, bloqueio de escrita para aluno e visitante, CRUD completo de usuários, paginação, e-mail duplicado e autoexclusão. Rodar `npm test` antes de mexer no app evitou depurar no celular um problema que era do servidor.

4.7 *[grupo]* Organização do time

Descreva aqui a divisão de tarefas, as ferramentas de acompanhamento usadas e o que o grupo faria diferente numa próxima fase.

---

5. Mapeamento dos requisitos

#	Requisito do enunciado	Onde foi atendido
1	Página principal com lista de posts, título/autor/descrição e busca	`mobile/src/screens/posts/PostListScreen.js`
2	Página de leitura do post	`mobile/src/screens/posts/PostDetailScreen.js`
3	Página de criação de postagens	`mobile/src/screens/posts/PostFormScreen.js` (modo criação)
4	Página de edição de postagens	`mobile/src/screens/posts/PostFormScreen.js` (modo edição)
5	Criação de professores	`UserFormScreen` via rota `ProfessorForm`
6	Edição de professores	`UserFormScreen` via rota `ProfessorForm` com `id`
7	Listagem paginada de professores com editar/excluir	`UserListScreen` via rota `ProfessorList` + `Paginador`
8	Mesmas telas para estudantes	Rotas `AlunoList` / `AlunoForm`
9	Página administrativa de posts	`mobile/src/screens/admin/AdminPostsScreen.js`
10	Autenticação e autorização	`AuthContext`, `LoginScreen`, middlewares `autenticar` / `autorizar`
T1	React Native com hooks e componentes funcionais	Todo o `mobile/src`
T2	Estilização	`mobile/src/theme` + componentes de UI
T3	Integração REST (posts, alunos, professores, auth) e permissões	`mobile/src/api/services.js` + rotas do backend
T4	Documentação técnica	README e este documento